Parallel Feedforward Neural Network

Rowan Li Tangalin (ID: 242920)
Email: rowanli.tangalin@studenti.unitn.it
Course: Introduction to Parallel Computing (2025–2026)

***Abstract*—Dense matrix products account for much of the work in fully connected neural network training. How quickly they run depends on cache locality, the amount of parallel work available, and the synchronization required between workers. This project studies those effects using a small two-layer sigmoid network implemented in four ways: direct sequential, tiled sequential, OpenMP, and replicated-model Message Passing Interface (MPI). The study contains 525 timed runs covering 105 configurations on individual processor cluster nodes. For both tested workloads, the tiled sequential implementation with packed transposes ran about three times faster than the direct version. OpenMP produced a self-speedup of 10.58 with 32 threads on the canonical problem and 24.43 with 64 threads on the large-input problem. At 64 ranks, MPI reached a self-speedup of 20.55; however, its per-rank batch convention also enlarged the global batch and lowered the number of updates in each epoch. Further weak scaling, tile-size, and batch-size experiments show that the interpretation depends strongly on how the workload and configuration are changed. These measurements describe single-node system performance, not improvements in model quality or scaling across multiple nodes.**

***Index Terms*—parallel computing, neural-network training, OpenMP, MPI, cache tiling, strong scaling, weak scaling**

I. Introduction

Problem and importance. Forward propagation, backpropagation, and the construction of weight gradients all require dense matrix multiplication. Counting floating-point operations alone is therefore not enough to explain runtime: data locality, the shape of the parallel work, and synchronization also matter. A compact C implementation makes these factors visible and keeps the connection between the algorithm and its execution straightforward.

Objectives. Four research questions (RQs) guide the experiments. RQ1 measures the gain from the optimized sequential path over the direct baseline. RQ2 examines OpenMP and single-node Message Passing Interface (MPI) scaling on fixed nominal problems. RQ3 compares the behaviour produced by two different weak-scaling rules. Finally, RQ4 studies tile size and the different batch conventions used by the backends.

Contribution. The same training pipeline is implemented in four forms, with ownership, synchronization points, and timing boundaries stated explicitly. The contribution lies in the implementation and interpretation of the resulting performance data. It is not a proposal for a new optimizer, network architecture, or distributed-training method.

II. State of the Art

Blocking and operand packing have long been used to improve locality in dense matrix multiplication [1], [2]. OpenMP provides thread teams, loop work sharing, barriers, and single-instruction, multiple-data (SIMD) constructs [3], while MPI defines the collectives used here to distribute data and sum gradients [4]. Full-scale data-parallel frameworks also keep model replicas and synchronize their gradients. PyTorch Distributed Data Parallel, for example, adds techniques such as gradient bucketing and overlap between communication and computation [5]. Such systems provide context, but they are not equivalent performance baselines for this small implementation.

This work intentionally considers a narrower question. It records which worker owns an output tile, whether a stated batch size applies globally or to each rank, and where synchronization occurs. These choices are necessary for correctness, and they also determine what the scaling plots mean. Keeping the implementation small exposes those choices instead of delegating them to a framework.

III. Contribution and Methodology

A. Model, Symbols, and Data

Let an actual mini-batch contain B samples. If the input, hidden, and output widths are I, H, and O, respectively, its forward pass is

$$\begin{aligned} Z_h &= XW_h^{\mathsf{T}} + \mathbf{1}_B b_h^{\mathsf{T}}, & H_a &= \sigma(Z_h), \\ Z_o &= H_a W_o^{\mathsf{T}} + \mathbf{1}_B b_o^{\mathsf{T}}, & \widehat{Y} &= \sigma(Z_o). \end{aligned} \tag{1}$$

In these expressions, $X \in \mathbb{R}^{B\times I}$ is the packed input, $W_h \in \mathbb{R}^{H\times I}$ and $W_o \in \mathbb{R}^{O\times H}$ are output-major weight matrices, b_h and b_o are the bias vectors, Z denotes a pre-activation, and σ applies the sigmoid element by element. Given target matrix T, backpropagation computes

$$\begin{aligned} D_o &= (\widehat{Y} - T) \odot \sigma'(Z_o), & D_h &= (D_o W_o) \odot \sigma'(Z_h), \\ \nabla W_o &= D_o^{\mathsf{T}} H_a, & \nabla W_h &= D_h^{\mathsf{T}} X. \end{aligned} \tag{2}$$

D represents the loss derivative with respect to a pre-activation, $\odot$ denotes element-wise multiplication, and T denotes a transpose. Summing the corresponding rows of D_o and D_h gives the bias gradients.

The reported objective is the mean half-squared training loss,

$$L = \frac{1}{2N} \sum_{n=1}^{N} \|\widehat{y}_n - y_n\|_2^2. \tag{3}$$

The implementation accumulates gradient sums and then updates the parameters using $\theta \leftarrow \theta - (0.5/B_a)\nabla\theta$, where B_a is

Algorithm 1 High-level mini-batch training pipeline.

Require: dataset D, model M, maximum batch size $B_{\max}$, epochs E
Ensure: updated model M
  procedure TRAIN($D, M, B_{\max}, E$)
    for $e \leftarrow 1$ **to** E **do**
      for all consecutive mini-batches Q in D **do**
        $(X, T) \leftarrow$ PREPAREBATCH(Q)
        $(\widehat{Y}, S) \leftarrow$ FORWARD(M, X)
        $G \leftarrow$ BACKWARD($M, S, \widehat{Y}, T$)
        $M \leftarrow$ UPDATE($M, G, 0.5/|Q|$)
      end for
    end for
    return M
  end procedure

Algorithm 2 High-level replicated-model MPI training.

Require: root dataset D, initial model specification M_0, local batch size B_{local}, epochs E
Ensure: synchronized model M on every rank
  procedure DISTRIBUTEDTRAIN($D, M_0, B_{\text{local}}, E$)
    $D_r \leftarrow$ BALANCEDSCATTER(D)
    $M \leftarrow$ IDENTICALMODEL(M_0)
    for $e \leftarrow 1$ **to** E **do**
      for all synchronized local mini-batches Q_r in D_r **do**
        $G_r \leftarrow$ BATCHGRADIENTSUM(Q_r, M)
        $B_g \leftarrow$ ALLREDUCESUM($|Q_r|$)
        $G \leftarrow$ ALLREDUCESUM(G_r)
        $M \leftarrow$ UPDATE($M, G, 0.5/B_g$)
      end for
    end for
    return M
  end procedure

the actual sequential/OpenMP batch size or the global batch size summed across MPI ranks.

The synthetic inputs are deterministic and lie in $[0, 1]$. Each target is a one-hot vector whose active position corresponds to the largest among the first O input features. Weights are drawn from the Xavier-uniform distribution [6], and all biases start at zero. The dataset generator is seeded with 12345; immediately before constructing the model, each active driver sets the seed to 12345 again. Timed repetitions clone a common master model. Samples retain ascending order throughout, with no shuffling. Parameters, gradients, and the eight reusable batch-workspace arrays are stored as single-precision values aligned to 64-byte boundaries.

B. Mini-Batching

Training walks through consecutive mini-batches and never shuffles the samples. When the dataset size is not divisible by $B_{\max}$, the final batch is smaller. Workspace memory is allocated once and then reused. Different batches cannot be processed concurrently because an update changes the parameters seen by the following batch. OpenMP parallelism is consequently applied to regular operations *inside* a batch. Since gradients are sums, the normalization uses the batch's actual size B, including for the short final batch, rather than always using $B_{\max}$. The arithmetic is already defined by (1) and (2), so Algorithm 1 keeps only the order of the main stages.

C. MPI General Parallel Framework

The MPI version follows synchronous data parallelism. Rank 0 divides the dataset into balanced, contiguous blocks and scatters one block to each rank. Every process holds a full model replica with identical initial parameters, then forms gradient sums from its local data. Here, the command-line batch size is *per rank*, not global. Before training starts, ranks check that they will execute the same number of updates. Without this check, they could enter the blocking collectives in different orders. The algorithm listing treats the four parameter-gradient arrays as one logical gradient object.

Local matrix operations on an MPI rank use the sequential runtime-tiled kernels. Doing so keeps the experiment focused on process-level MPI and avoids silently introducing a hybrid MPI+OpenMP implementation, while still giving each rank an optimized local kernel. Every update performs one blocking `Allreduce` for the sample count and four more for the gradient arrays. Thus, the single gradient-sum operation shown in Algorithm 2 expands to four calls in the C implementation. The buffers are not fused, and computation does not overlap communication. Worker-count comparisons remain useful, but the OpenMP and MPI wall times should not be read as a direct ranking: their batch meanings and job placements differ.

D. Tiled Matrix Multiplication

The optimized base general matrix multiplication (GEMM) kernel accumulates AB into C. Two packed variants perform the corresponding operations for AB^{T} and $A^{\mathsf{T}}B$. A caller requesting a new product clears C first. Loop bounds divide the matrices logically into edge-safe tiles with runtime width T; the base kernel does not allocate and copy each tile into independent storage. Unless an experiment selects another value, $T = 64$. Each output tile has exactly one thread owner in the OpenMP kernels, and that owner covers the complete reduction dimension. Depending on the number of threads and output tiles, a thread can receive no tiles, one tile, or several. Two threads never update the same output tile, so the multiplication needs neither atomics nor a reduction over private partial results. Algorithm 3 summarizes the ownership and reduction order.

In the OpenMP kernel, `collapse` combines the two output-tile loops into one work-sharing space. The dense tiles have broadly similar costs, so `schedule(static)` assigns them without paying dynamic-scheduling overhead. The innermost, contiguous j loop is marked for SIMD execution. Before multiplication, a transpose variant packs the required logical transpose into an aligned buffer that can

Algorithm 3 Tile-level view of the blocked $C \mathrel{+}= AB$ operation.

Require: compatible matrices A, B, and C; tile width T
Ensure: C is increased by AB
  procedure TILEDGEMM(A, B, C, T)
    Assign each output tile C_{ij} to exactly one thread
    for all output tiles C_{ij} owned by the current thread **do**
      for all reduction tiles k **do**
        $C_{ij} \leftarrow C_{ij} + A_{ik}B_{kj}$
      end for
    end for
    return C
  end procedure

be reused. Packing and multiplication run inside the same OpenMP team; the implicit barrier between them prevents a thread from consuming an unfinished buffer. Static scheduling is also used for the other regular row and array loops. Bias gradients require a different approach: every thread builds a private partial vector, after which columns are folded in increasing thread-ID order. This removes atomics and fixes the summation order for a given team.

For E epochs and N samples, sequential work is $\Theta(EN(IH + HO))$. MPI performs the corresponding local work with N_r, the sample count assigned to rank r. Its four gradient arrays contain $HI + H + HO + O$ floats and are reduced collectively on every update; the `Allreduce` implementation determines the actual transport volume. With p ranks and a local batch bound B, the typical global batch is close to pB, up to the dataset size. Raising the rank count can therefore lower the number of optimizer updates as well as divide the local computation.

IV. Experiments and System Description

A. Platform

Every Portable Batch System (PBS) job requested a single node from UniTrento's HPC3 cluster. GNU Compiler Collection (GCC) 13.3.0 compiled the sequential and OpenMP programs. The MPI build used GCC 13.2.0 together with OpenMPI 4.1.6. All variants were compiled as C11 code with Portable Operating System Interface (POSIX).1-2001 features, `-O3`; `-fopenmp` was added where required. OpenMP jobs requested close binding to cores and interleaved non-uniform memory access (NUMA) allocation. MPI jobs mapped and bound ranks by core, using the shared-memory and self transports. PBS allocations ranged from one central processing unit (CPU) slot and 8 GB for the sequential experiment to 64 CPU slots and 32–128 GB for the parallel experiments. Commands run on a compute node report x86-64 architecture and four Intel Xeon Gold 6252N sockets at a nominal 2.30 GHz. Each socket has 24 physical cores, giving 96 cores in total, with one hardware thread per core and therefore no simultaneous multithreading (SMT). The machine exposes four NUMA domains, one per socket and 24 cores per domain, with approximately 188–189 GiB per domain (about 755 GiB reported in total).

TABLE I
EXPERIMENTAL DESIGN. HERE p IS THE WORKER COUNT.

Exp.	Purpose	Problem rule and sweep
1	sequential optimization	5000 canonical and 500 large-input samples; direct vs. tiled
2	OpenMP fixed problem	Exp. 1 workloads; 1–64 threads
3	MPI fixed nominal problem	5000 canonical, local batch 64; 1–64 ranks
4	OpenMP weak scaling	5000 samples, hidden width $128p$; 1–64 threads
5	MPI weak scaling	$500p$ canonical samples; 1–64 ranks
6	MPI tile sensitivity	5000 canonical; tile 16–2048 at 1, 16, and 64 ranks
7	batch sensitivity	8192 canonical; batch 16–512 for sequential, OpenMP at 8/32/64 threads, and MPI at 8/32/64 ranks

Each core has separate 32 KiB L1 data and instruction caches and a private 1 MiB L2 cache; each socket has a shared 35.8 MiB L3 cache, for 143 MiB across the node. Cache lines are 64 bytes. The captured operating system is GNU/Linux with kernel `5.14.0-611.36.1.el9_7.x86_64`. Runtime clock frequency and the turbo-frequency policy were not recorded, so 2.30 GHz is only the processor's nominal model frequency.

B. Design and Metrics

Two network shapes are used. The canonical case is $1024 \to 1536 \to 10$, whereas the large-input case is $178608 \to 448 \to 10$. The seven experiment groups appear in Table I. Batch sizes refer to the complete batch in the sequential and OpenMP programs, but to one rank's local batch in MPI. Each configuration has two warmups followed by five timed repetitions. The current logs therefore contain 525 measurements from 105 configurations.

The training timer begins after model and dataset construction and after the trainer allocates its eight batch-workspace arrays. There is one subtle exception. The transpose-packing and OpenMP bias-reduction helpers enlarge reusable static buffers lazily, so their first allocation occurs inside the first epoch's timed region. In normal runs, the two warmups establish the necessary capacities before recorded repetitions. The timed region includes batch packing, forward and backward computation, parameter updates, OpenMP team and barrier costs, and the MPI collectives performed on each update. Optional epoch-loss computation begins only after that epoch's timer has stopped. MPI records the largest training time across ranks because the slowest rank determines completion. A separate timer covers the final loss evaluation.

The central runtime reported for a configuration is its *median*, or 50th percentile. Spread is represented by the *interquartile range* (IQR), $Q_3 - Q_1$, which covers the middle half of the observations. These rank-based statistics are comparatively resistant to an isolated slow run and do not assume normally distributed timings [7]. Only five timed repetitions are available, so the IQR should be read as a description of spread, not as a confidence interval or a significance test.

For a fixed workload, speedup is $S_p = T_1/T_p$ and parallel efficiency is $\eta_p = S_p/p$ [8]. With the proportional-growth

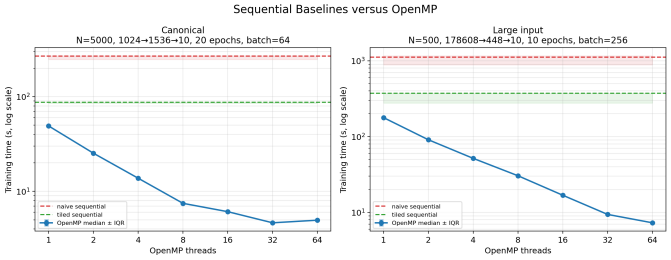

Fig. 1. Sequential baselines and OpenMP scaling for the matched workloads. Points and bands report median and IQR.

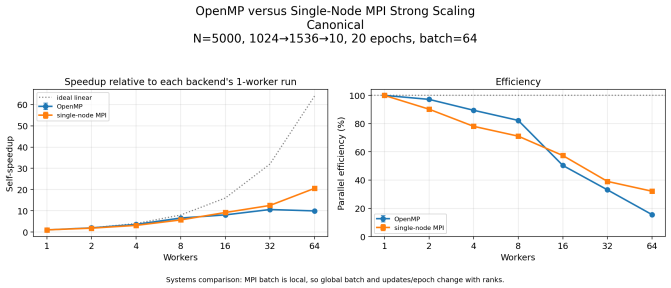

Fig. 2. Backend-relative speedup and efficiency. MPI's global batch and update count change along its rank sweep.

rules used here, weak efficiency becomes T_1/T_p. The OpenMP and MPI weak-scaling experiments grow different resources, however, and cannot be used to rank the two backends [9]. Speedups stated in the prose are ratios between median runtimes. Fig. 1 summarizes the sequential and OpenMP runtimes. The plotting code for Fig. 2 takes another valid route: it pairs runs with equal repetition identifiers, computes $T_{1,r}/T_{p,r}$ for each pair, and then plots the median and IQR of those ratios. Since sweep points were executed in ascending order rather than randomized, a gradual change in node conditions could influence them.

V. Results and Discussion

RQ1 - Does local optimization help? Yes. The improvement is roughly $3\times$ for both workloads. On the canonical problem, the direct sequential median was 269.184 s, compared with 87.423 s for the tiled implementation, a $3.08\times$ difference. The large-input medians were 1114.349 s and 371.809 s, respectively ($3.00\times$). Because tiling and packed-transpose handling were introduced together, this experiment cannot attribute a separate gain to either one.

RQ2 - How do OpenMP and MPI scale? For OpenMP, the answer depends on matrix shape. The fastest canonical result was 4.647 s with 32 threads, corresponding to a $10.58\times$ self-speedup. Increasing the team to 64 threads raised the time to 4.953 s. In contrast, the large-input workload continued to improve through 64 threads, where it reached 7.289 s and $24.43\times$. The useful GEMM work is determined by the number of tiles in the actual $R \times C$ output: $\lceil R/64 \rceil \lceil C/64 \rceil$. A small batch or a narrow output can therefore leave some threads without a tile even if the reduction dimension is large. The ordinary and transpose-B kernels produce an $M \times N$ output, while the transpose-A kernel produces $K \times N$.

MPI time decreased from 73.683 s at one rank to 3.586 s at 64 ranks, giving $20.55\times$ speedup and 32.11% efficiency. That result does not demonstrate that MPI is faster than OpenMP. The jobs ran on different nodes, and the MPI sweep did not preserve the training schedule: its effective global batch rose from 64 to 4096 while updates per epoch dropped from 79 to 2. The curve accurately describes the implemented configuration, but it is not strong scaling of a completely unchanged computation.

RQ3 - Do the weak-scaling rules keep runtime constant? Neither rule does. The OpenMP experiment increased hidden width with the thread count; runtime grew from 5.764 s to 33.661 s, leaving 17.12% endpoint efficiency. MPI retained approximately 500 samples per rank and increased from 7.458 s to 18.294 s, with 40.77% endpoint efficiency. These figures answer different questions. OpenMP increases the parameter count and arithmetic work, whereas MPI leaves the model unchanged, expands the dataset, and adds participants to every gradient collective. Comparing their endpoint efficiencies as if they ranked the backends would therefore be misleading.

RQ4 - Do tile and batch choices matter? They do, although for different reasons. Across the tested 1-, 16-, and 64-rank cases, tile 256 was 8.66–13.37% faster than tile 64. This identifies the best value among these configurations, but does not explain it. Without cache and instruction counters, reduced misses and lower loop overhead remain plausible explanations rather than measured causes. Larger batches usually shortened runtime by reducing the number of updates and amortizing fixed team or collective costs. With 64 MPI ranks, local batches from 16 to 512 produced global batches from 1024 to 8192 and reduced the epoch to between eight and one updates. Batch size may also alter how many steps are required to reach a chosen quality target [10], which was not measured in these experiments.

Quality check and limits. The final loss in Experiment 7 was close to $L = 0.5$. Under (3), an all-zero prediction against a one-hot target also gives exactly 0.5. The observed loss is consequently compatible with a trivial near-zero-output model. It supports comparisons of final training loss across configurations, but not a claim of successful classification. There is no validation set or recorded accuracy. Likewise, the timings do not separate cache, bandwidth, NUMA, computation, and synchronization costs, and every MPI measurement comes from a single node.

VI. Conclusions and Future Work

The measurements answer all four research questions, though each answer has a clear boundary. Packing and tiling make the sequential baseline about $3\times$ faster. The most effective OpenMP team size varies with workload shape. MPI lowers elapsed time while also changing both the global batch and the number of updates. Conclusions from the weak-scaling, tile, and batch sweeps depend on the rule used to change the configuration. These findings concern system

behaviour; they do not establish model quality or multi-node scalability.

Hardware-counter and profiling data would help separate cache, bandwidth, NUMA, computation, and synchronization effects. MPI should also be tested with a fixed global batch and a stated accuracy or loss target. For multiple nodes, a natural extension is a hybrid design using MPI between nodes or NUMA domains and OpenMP within each rank. Pure OpenMP, pure MPI, and hybrid runs should then use equal core counts, datasets, global batches, and placements. Finally, fused gradient buffers and nonblocking collectives would permit communication overlap to be measured rather than only suggested.

VII. Reproducibility and Artifact Availability

Git: https://github.com/SSorbo/PARCO-Computing-2026-242920

Artifact layout. C sources and headers are stored in `src/` and `include/` respectively; the Makefile is at the project root. The `pbs_scripts/` directory contains the eight experiment jobs, `artifacts/` retains their captured standard-output and error logs, and `results/` contains the raw CSV measurements. Plotting programs and their Python dependencies are provided in `plot_scripts/` and `requirements.txt`; generated figures are stored under `plots/`.

Build and verification. Starting in `Project Code/`, `make seq`, `make omp`, and `make mpi` build the three executables. The commands `make test` and `make test-mpi` run the shared-memory and MPI correctness suites; the latter requires `mpicc` and `mpirun`. The Makefile records the common C11, POSIX, optimization, warning, OpenMP, and math-library options used by these builds.

Reproducing measurements. Jobs must be submitted from `Project Code/` with `qsub pbs_scripts/<job>.pbs`. The README maps every experiment to its submission script and output CSV. Each script records the complete network shape and sweep, two warmups, five timed repetitions, and the relevant binding settings. OpenMP jobs set `OMP_PROC_BIND=close` and `OMP_PLACES=cores`; MPI jobs use core binding and `UCX_TLS=sm,self`. The program fixes the dataset and model seeds at 12345. Rerunning a job recreates its corresponding file in `results/`, while scheduler logs are written to `artifacts/`.

Regenerating figures. Python dependencies are installed with `python3-mpipinstall-rrequirements.txt`. From `Project Code/`, the two figures used in this report are regenerated with `python3plot_scripts/plot_cross_experiment.py--results-dirresults--outdirplots/cross_experiment`. The README gives the analogous commands for each experiment-local plot and the batch-sensitivity figures. Plotting uses the raw repetitions to report medians and interquartile ranges; no values are copied manually into the figures.

References

[1] M. D. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS IV)*, 1991, pp. 63–74, doi: 10.1145/106973.106981.

[2] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Trans. Math. Softw.*, vol. 34, no. 3, Art. no. 12, pp. 1–25, 2008, doi: 10.1145/1356052.1356053.

[3] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, Nov. 2021. [Online]. Available: https://www.openmp.org/spec-html/5.2/openmp.html

[4] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard, Version 4.1*, Nov. 2023. [Online]. Available: https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report/mpi41-report.htm

[5] S. Li *et al.*, "PyTorch Distributed: Experiences on accelerating data parallel training," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3005–3018, 2020, doi: 10.14778/3415478.3415530.

[6] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 13th Int. Conf. Artificial Intelligence and Statistics*, PMLR, vol. 9, 2010, pp. 249–256.

[7] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*, 2015, Art. no. 73, pp. 1–12, doi: 10.1145/2807591.2807644.

[8] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485, doi: 10.1145/1465482.1465560.

[9] J. L. Gustafson, "Reevaluating Amdahl's law," *Commun. ACM*, vol. 31, no. 5, pp. 532–533, May 1988, doi: 10.1145/42411.42415.

[10] C. J. Shallue, J. Lee, J. Antognini, J. Sohl-Dickstein, R. Frostig, and G. E. Dahl, "Measuring the effects of data parallelism on neural network training," *J. Mach. Learn. Res.*, vol. 20, no. 112, pp. 1–49, 2019.